SEMANA 13 · ARQUITECTURA EN LA NUBE

OpenStack II: Arquitectura Interna, Redes de Producción y Alta Disponibilidad

Parte 1 de 4 · Anatomía y Ciclo de Vida del Aprovisionamiento

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Parte 1 de 4 · Anatomía y Ciclo de Vida del Aprovisionamiento

Sistema Desacoplado Asíncrono

Servicios coordinados por bus de mensajes y base de datos



Keystone

identidad y tokens



Nova API

recibe peticiones de cómputo



Neutron

redes virtuales



Cinder

volúmenes persistentes

mensaje asíncrono



RabbitMQ: cola de mensajes

Protocolo de colas avanzado (AMQP)

Nadie llama directo a otro servicio



MariaDB Galera: base en clúster

Guarda el estado global

API = interfaz de programación

Fuente: docs.openstack.org/nova/latest/user/architecture.html

Keystone y Tokens Fernet

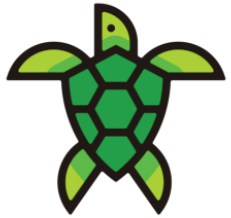
Validación criptográfica simétrica sin consultas a base de datos



Cliente CLI

(línea de comandos)

credenciales →



Keystone

emite un token Fernet

token cifrado →



Nova API

valida el token en memoria

Contenido del token cifrado

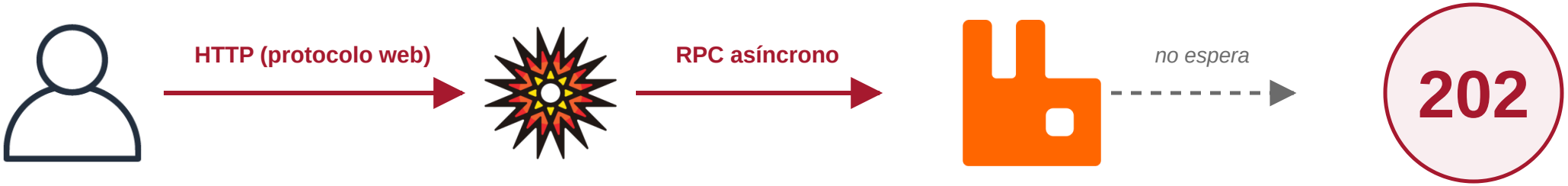
usuario	proyecto activo	roles asignados

CLI = interfaz de línea de comandos · API = interfaz de programación

Fuente: docs.openstack.org/keystone/latest/admin/identity-tokens.html

Nova API: Recepción de Solicitud

Validación de cuotas de proyecto y delegación asíncrona hacia el bus



Petición HTTP

POST /servers

con token válido

1. Valida cuotas de vCPU y RAM
2. Registra la VM en estado BUILD

Cola Nova Conductor

El cliente recibe confirmación inmediata.

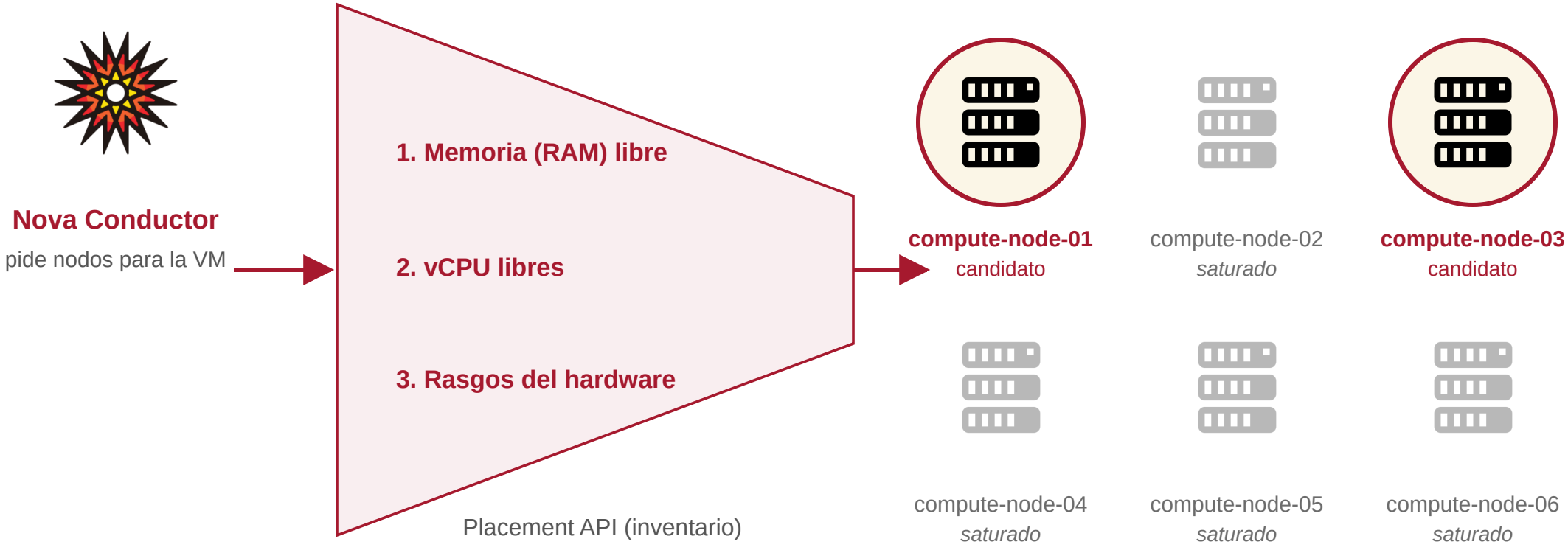
El cómputo y el disco se ejecutan después, en segundo plano.

HTTP = protocolo web · vCPU = núcleo virtual asignado a una instancia · RAM = memoria de acceso aleatorio · VM = máquina virtual · RPC = llamada a procedimiento remoto · API = interfaz de programación

Fuente: docs.openstack.org/nova/latest/user/architecture.html

Placement API: Filtrado de Inventario

Clasificación de hipervisores disponibles según memoria, vCPUs y rasgos



Placement no decide el destino: solo entrega candidatos.

VM = máquina virtual · vCPU = núcleo virtual asignado a una instancia · API = interfaz de programación

Fuente: docs.openstack.org/placement/latest/

Nova Scheduler (asignador de Nova): pesaje y elección

Cada hipervisor (servidor físico) recibe un puntaje; gana el más alto.

RAM libre por nodo (pesaje)

compute-node-01

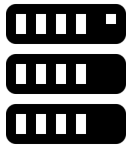


16 GB

compute-node-03



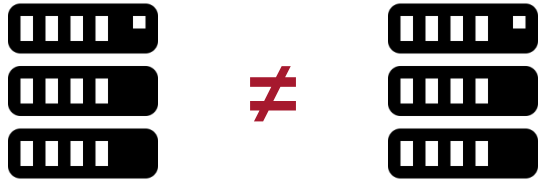
Más RAM libre = más puntaje



Destino: compute-node-03

puntaje más alto

Anti-afinidad



Dos instancias del mismo servicio
no comparten equipo físico



Nova Conductor

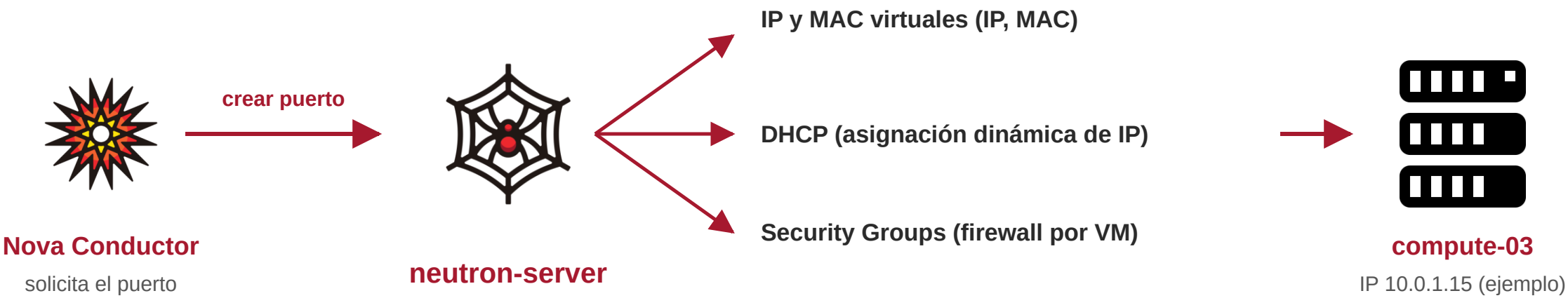
coordina la reserva

RAM = memoria de acceso aleatorio · GB = gigabyte

Fuente: docs.openstack.org/nova/latest/user/filter-scheduler.html

Neutron: Reserva de Puerto Virtual e IP

Creación del puerto lógico, asignación DHCP y enlace de seguridad



Port binding: reservado en el host



Sin reserva de red no hay arranque.

IP = dirección de red · MAC = dirección física de la tarjeta · DHCP = asignación dinámica de direcciones · VM = máquina virtual · CPU = unidad central de procesamiento · RAM = memoria de acceso aleatorio

Fuente: docs.openstack.org/neutron/latest/admin/intro-basic-networking.html

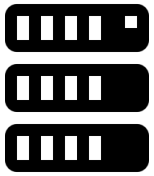
Glance y Cinder: Modalidades de Disco

Elección entre almacenamiento efímero local y volúmenes SAN persistentes

¿Los datos sobreviven al fallo del servidor?

No

Sí



Disco efímero

copia local en el hipervisor

Máximo I/O local en NVMe
Se borra con la VM
Ideal para apps sin estado

Regla: según la criticidad de los datos



Volumen Cinder

SAN o clúster Ceph (persistente)

Sobrevive a la caída del nodo
Permite migración en vivo
Datos críticos

VM = máquina virtual · SAN = red de área de almacenamiento

Fuente: docs.openstack.org/cinder/latest/

Nova Compute: Ejecución en KVM

Traducción XML en Libvirt, enlace de interfaz TAP y arranque QEMU



Nova Compute (demonio del hipervisor)

genera el XML (lenguaje de marcado extensible)



libvirt: API de gestión del hipervisor

traduce la orden a la pila



KVM: virtualización en el kernel

aprovecha el hardware del procesador



QEMU: emulador de la VM

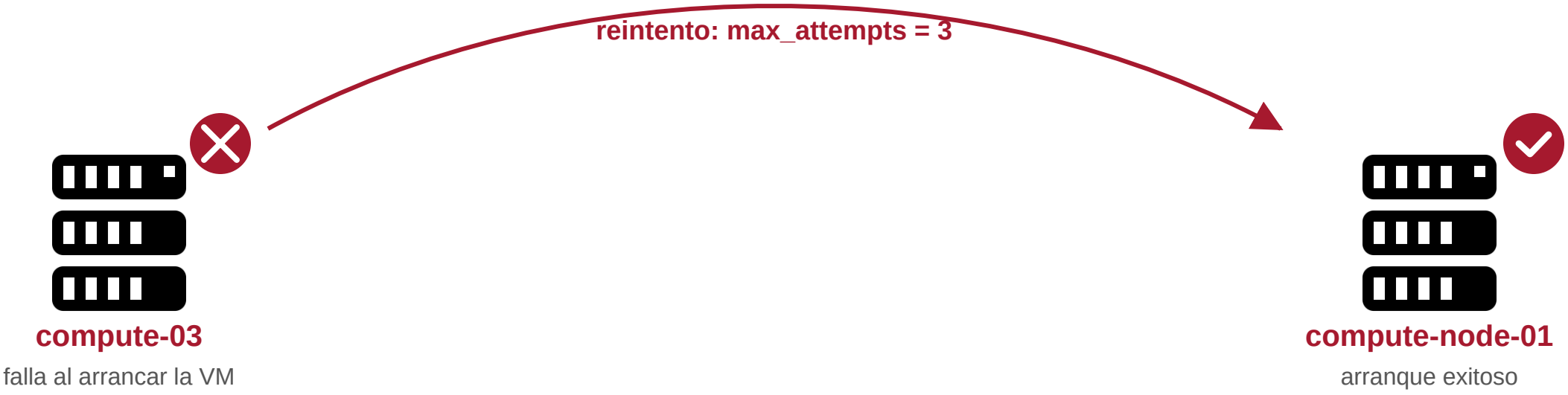
conecta TAP y disco

XML = lenguaje de marcado extensible · API = interfaz de programación · KVM = virtualización en el kernel · QEMU = emulador de hardware · VM = máquina virtual · TAP = interfaz de red virtual

Fuente: docs.openstack.org/nova/latest/admin/configuration/hypervisors.html

Manejo de Errores y Rollback

Detección de excepciones locales y re-planificación automática sin fugas



1 **intento fallido**
compute-03 sale de la lista

2 **segundo intento**
compute-node-01 recibe la VM

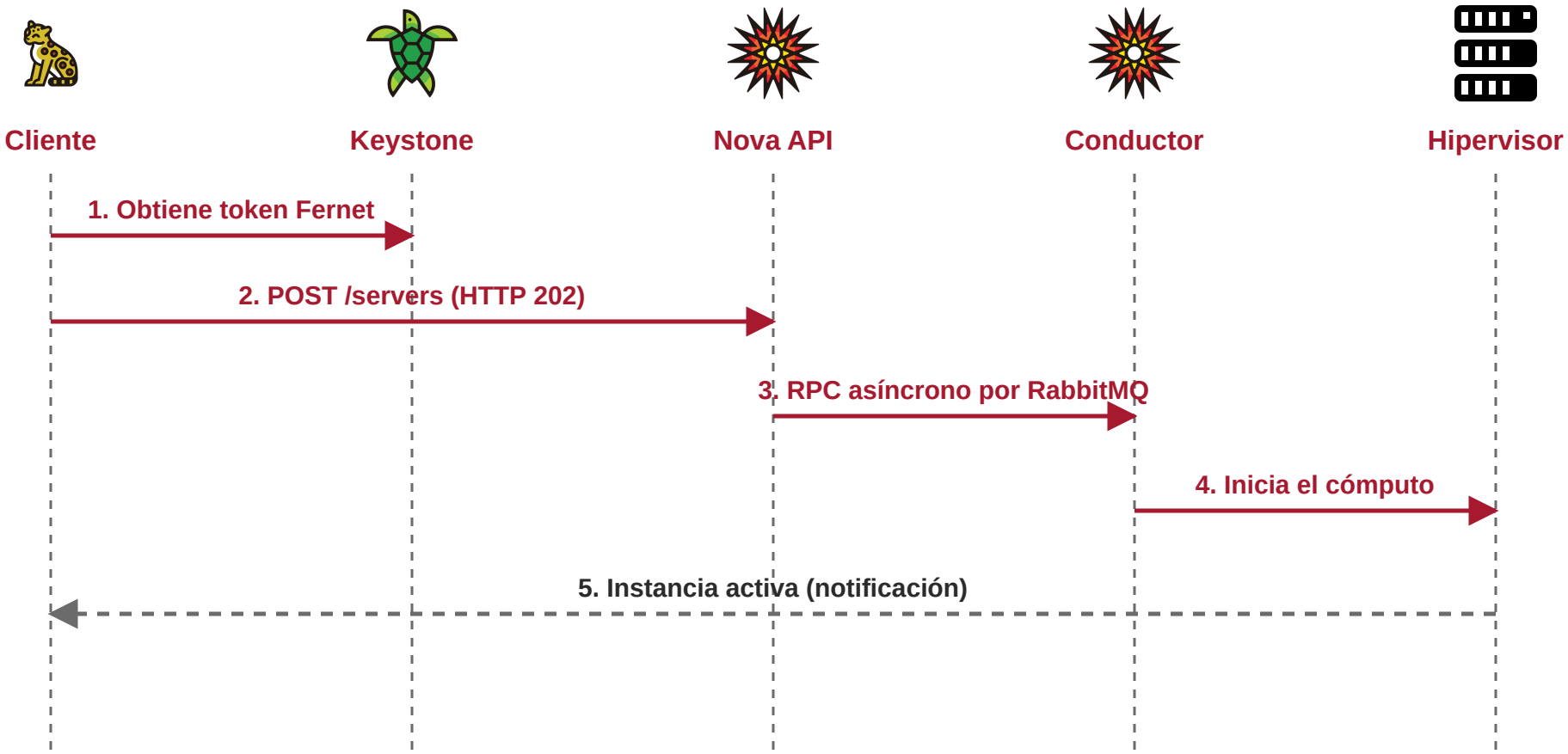
El usuario no ve la falla; todo queda en bitácoras.

VM = máquina virtual

Fuente: docs.openstack.org/nova/latest/user/architecture.html

Secuencia Completa de Aprovisionamiento

Cronología de eventos entre cliente, APIs, colas y el hipervisor



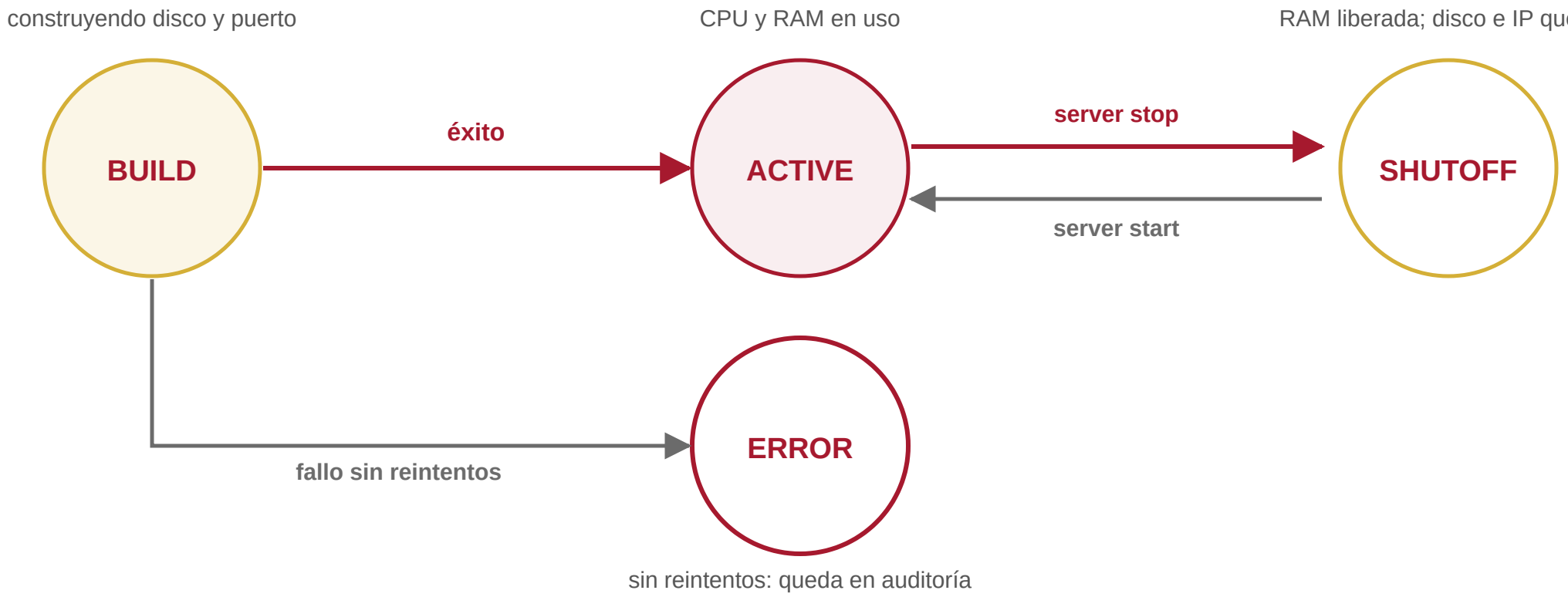
Cada flecha puede ser asíncrona: el emisor no espera la respuesta.

API = interfaz de programación · HTTP = protocolo web · RPC = llamada a procedimiento remoto

Fuente: docs.openstack.org/nova/latest/user/architecture.html

Máquina de Estados de la Instancia

Transición formal de estados y liberación de recursos de cómputo



CPU = unidad central de procesamiento · RAM = memoria de acceso aleatorio · IP = dirección de red

Fuente: docs.openstack.org/nova/latest/reference/vm-states.html

CASO ILUSTRATIVO

Caso de Estudio: Dimensionamiento de Cómputo

Caso ilustrativo, no es una empresa real: despliegue de 50 instancias empresariales



4 nodos de 64 núcleos

2.048 GB de RAM en total

768 vCPU sobre 256 núcleos (3:1, decisión de diseño)

50 VMs (máquinas virtuales)

10 BD	25 web	15 micro
bases de datos	servidores web	microservicios

Reglas: afinidad NUMA y anti-afinidad

256 núcleos y 2 TB de RAM

GB = gigabyte · RAM = memoria de acceso aleatorio · vCPU = núcleo virtual asignado a una instancia · BD = base de datos · NUMA = acceso a memoria no uniforme · TB = terabyte

Fuente: modelo de referencia de cómputo privado 2026 · Foto: Florian Hirzinger, CERN (CC BY-SA 3.0)

PRINCIPIOS GUÍA

Cuatro Principios del Ciclo de Vida

Directrices fundamentales para la arquitectura y operación de cómputo









1

Todo por API y bus

sin acoplamiento entre servicios

2

Controladores sin estado

el estado vive en MariaDB Galera

3

Tokens criptográficos

validación Fernet en memoria

4

Compensación ante fallas

rollback automático, sin huérfanos

Estos cuatro principios explican cada decisión de cómputo.

API = interfaz de programación

Fuente: OpenStack Architecture Design Guide 2026

TRANSICIÓN TEMÁTICA

Hacia la Parte 2: Redes de Producción

Del aprovisionamiento de cómputo hacia el plano de red definido por software

Dominado en la parte 1



Desacoplamiento por RabbitMQ



Planificación en Placement



Discos efímeros frente a Cinder

Contenido de la parte 2



Redes provider frente a self-service



Arquitectura SDN con OVN



Enrutamiento distribuido (DVR)

De aprovisionar cómputo a definir la red del centro de datos.

SDN = red definida por software · OVN = red virtual abierta

SEMANA 13 · ARQUITECTURA EN LA NUBE

OpenStack II: Arquitectura Interna, Redes de Producción y Alta Disponibilidad

Parte 2 de 4 · Redes Avanzadas de Producción: Provider Networks y OVN

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Parte 2 de 4 · Redes Avanzadas de Producción: Provider Networks y OVN
OVN = Open Virtual Network (red virtual abierta)

Provider Networks: Acceso L2 Físico

Mapeo directo de máquinas virtuales a VLANs y switches del centro de datos



VM (máquina virtual)
vNIC eth0 (tarjeta virtual)

Bridge OVS
br-ex (Open vSwitch)

Switch ToR
parte superior del rack

Red física
centro de datos

Sin NAT ni overlays

Latencia mínima

Cargas heredadas (misma subred)

VM = máquina virtual · vNIC = tarjeta de red virtual · OVS = Open vSwitch (conmutador virtual) · ToR = switch del rack · TAP = interfaz de red virtual · VLAN = red local virtual · L2 = capa 2 (enlace) · NAT = traducción de direcciones

Fuente: docs.openstack.org/neutron/latest/admin/intro-basic-networking.html

Self-Service Networks: Redes de Proyecto

Redes virtuales privadas superpuestas creadas autónomamente por los usuarios



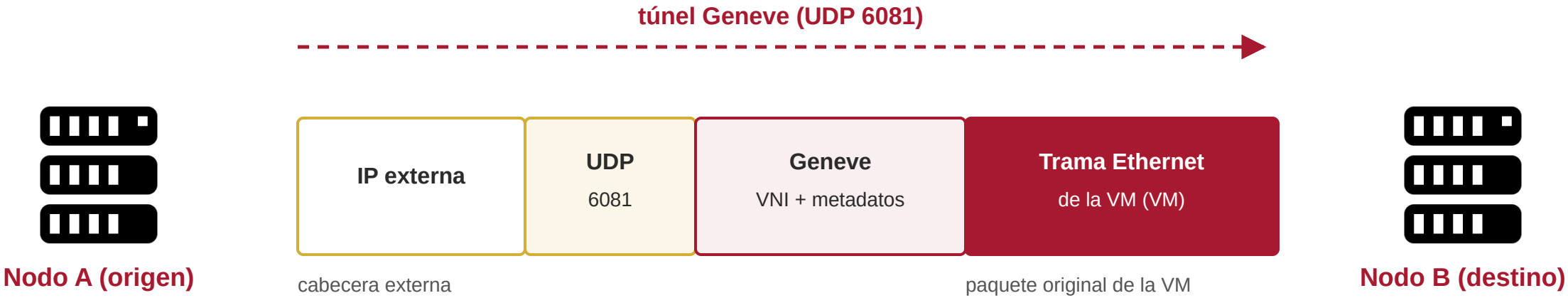
VNI (identificador de red virtual) separa cada proyecto.

L3 = capa 3 (red) · IP = dirección de red

Fuente: docs.openstack.org/neutron/latest/admin/intro-basic-networking.html

Túneles Geneve: Paquetes L2 sobre Red IP L3

Encapsulamiento con cabeceras extensibles para metadata de seguridad y enrutamiento



MTU 1600 en la red física (Jumbo Frames)

La cabecera Geneve agrega bytes al paquete.

MTU = unidad máxima de transmisión

UDP = protocolo de datagramas · IP = dirección de red · L2 = capa 2 (enlace) · L3 = capa 3 (red) · SDN = red definida por software

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

El Cuello de Botella Centralizado (ML2/OVS)

Enrutamiento L3 centralizado que concentraba fallas y latencia en nodos de red



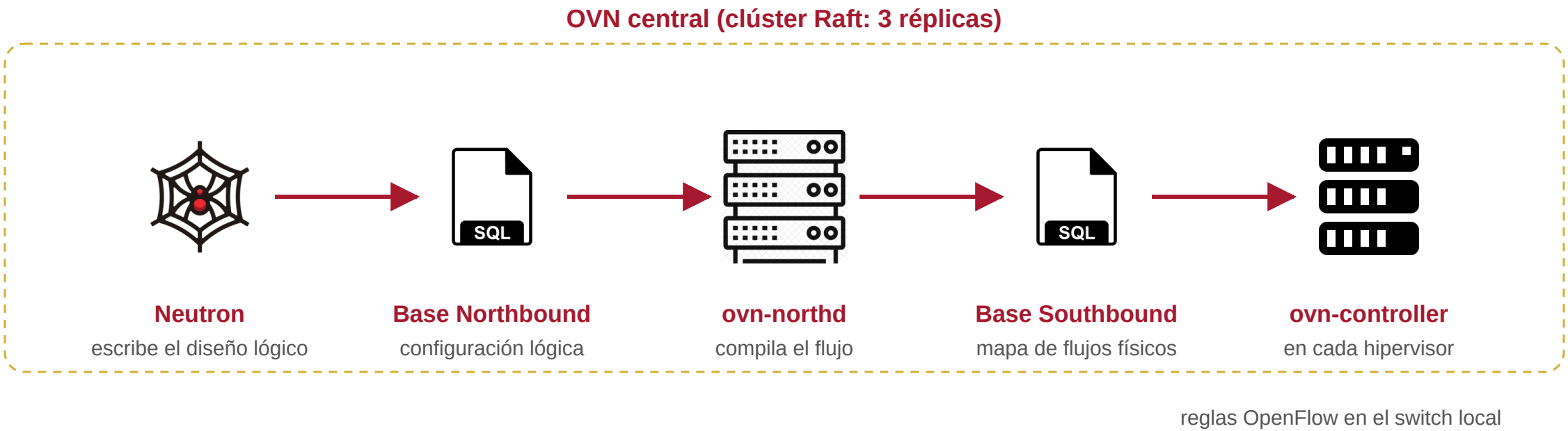
Si cae el nodo, se corta

L3 = capa 3 (red) · ML2 = plugin de red de Neutron · OVS = Open vSwitch (conmutador virtual)

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

Arquitectura OVN: Northbound y Southbound

Bases de datos lógicas distribuidas sincronizadas mediante ovsdb-server



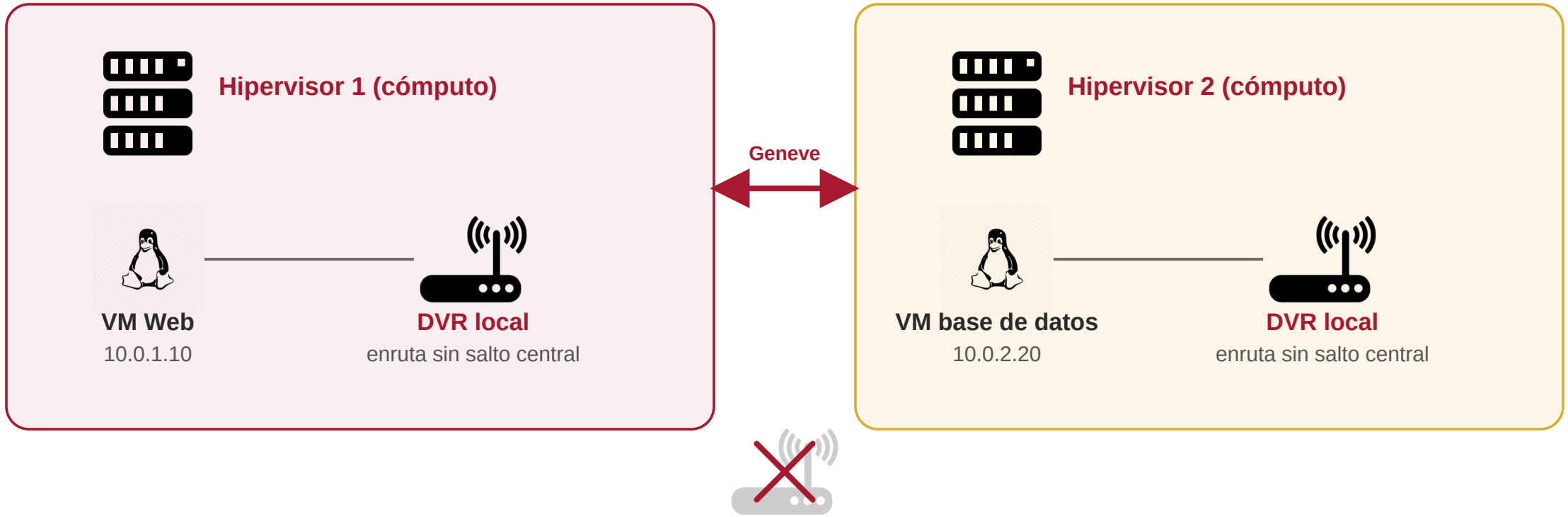
Declarativo: Neutron escribe el diseño

OVN = red virtual abierta

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

DVR: Enrutamiento en Cada Hipervisor

Enrutamiento L3 local directo sin saltos a servidores de red centrales



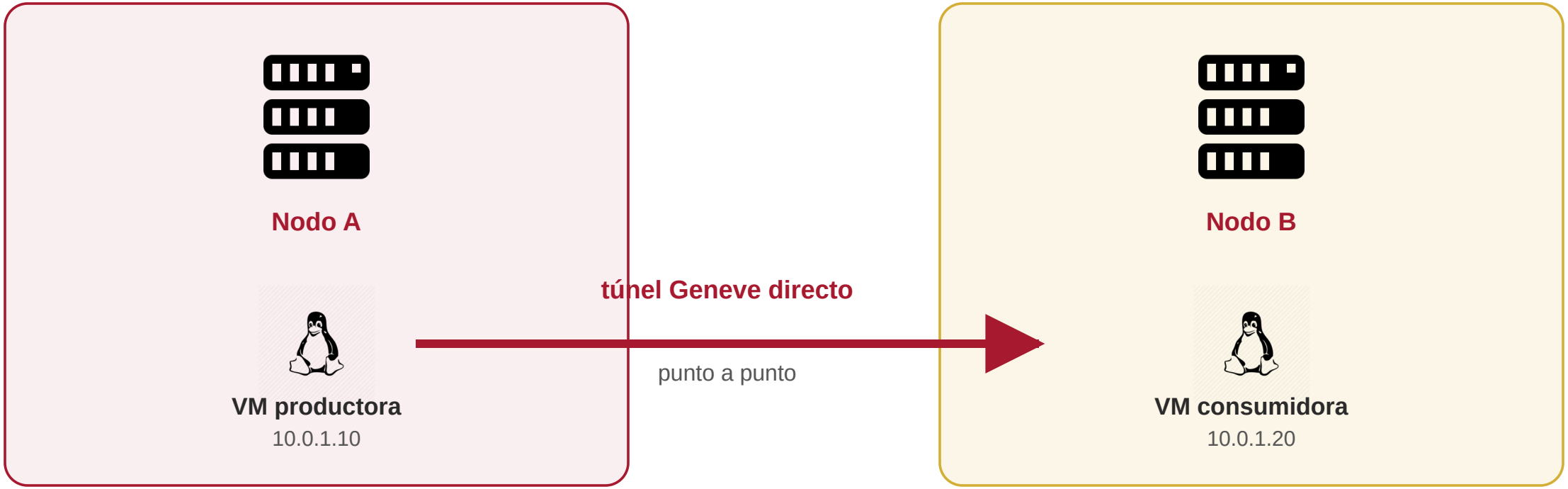
Sin nodo central: ningún router concentra el tráfico.

VM = máquina virtual · DVR = enrutamiento distribuido · L3 = capa 3 (red)

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

Tráfico Este-Oeste: Entre Servidores

Trayectoria directa punto a punto entre máquinas virtuales sin intermediarios



Tráfico entre VM del mismo centro de datos, sin nodo intermedio.

VM = máquina virtual

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

Tráfico Norte-Sur: Acceso a Internet

Salida distribuida directa o centralizada con Gateway SNAT



VM = máquina virtual · SNAT = traducción de direcciones de origen · IP = dirección de red · BFD = detección de fallas de reenvío

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

IP flotante (floating IP): mapeo estático 1 a 1

Una IP pública fija se asocia a una VM por regla 1 a 1

DNAT: 181.49.20.5 → 10.0.1.15

IP pública → IP fija



Usuario externo
llega a 181.49.20.5

Switch físico
entrega al hipervisor

Hipervisor
OVS (Open vSwitch) y DVR

VM
IP fija 10.0.1.15

Asociación estática 1 a 1

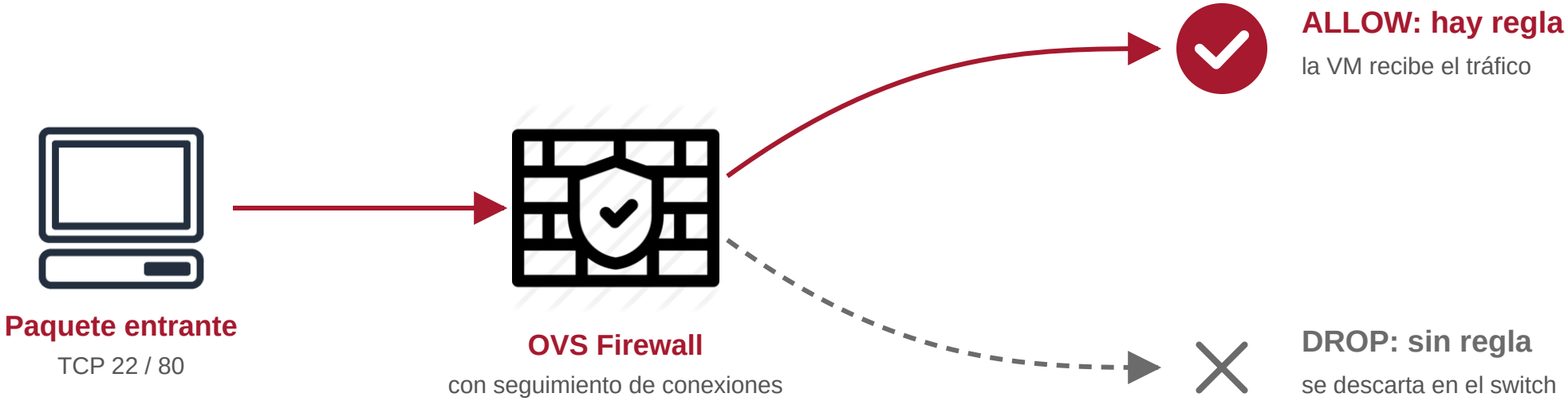
Sin nodos centrales

OVS = Open vSwitch (conmutador virtual) · DVR = enrutamiento distribuido · VM = máquina virtual · IP = dirección de red · DNAT = traducción de direcciones de destino

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

Security Groups: Filtrado en OpenFlow

Cortafuegos con inspección de estado directamente en el kernel mediante conntrack



Las respuestas de conexiones ya aceptadas pasan sin regla extra.

TCP = protocolo de control de transmisión · OVS = Open vSwitch (conmutador virtual) · VM = máquina virtual

Fuente: docs.openstack.org/neutron/latest/admin/config-ovs-firewall.html

Auditoría de Flujos Lógicos con ovn-trace

Simulación determinista del camino de red sin inyectar paquetes reales

ovn-trace red-privada

ip4.src = 10.0.1.15 · ip4.dst = 10.0.1.20

1. Switch lógico
de ingreso

2. Router L3
distribuido (capa 3)

3. Security Groups
ACL (lista de control)

FORWARDED
reenvía



Solo lee la base Southbound
no inyecta paquetes reales en la red



Indica el paso exacto
que reenvía o descarta el tráfico

L3 = capa 3 (red) · ACL = lista de control de acceso

Fuente: docs.openstack.org/neutron/latest/admin/ovn/index.html

CASO ILUSTRATIVO

Caso de Estudio: Topología de Tres Capas

Caso ilustrativo, no es una empresa real: arquitectura segura para producción



Defensa en profundidad

Solo capa 1 con IP pública

IP = dirección de red

Fuente: Topología de referencia para nube privada empresarial 2026

Provider Networks frente a Self-Service

Guía de decisión arquitectónica según casos de uso empresariales



Provider network



Self-service (OVN)

✓	Coexistencia con servidores físicos	○
✓	Latencia mínima, sin Geneve	○
✓	Cargas heredadas sin cambiar IP	○
○	Autonomía del desarrollador	✓
○	Espacios de IP solapados por proyecto	✓
○	Aislamiento multi-tenant estricto	✓

Regla: necesita L2 físico → provider; autonomía → self-service.

IP = dirección de red · L2 = capa 2 (enlace)

Fuente: OpenStack Networking Guide 2026

Hacia la Parte 3: Almacenamiento y HA

De las redes definidas por software hacia la resiliencia en almacenamiento y control

Dominado en la parte 2



Provider networks en VLAN

Túneles Geneve y overlays

OVN y enrutamiento DVR

Contenido de la parte 3



Clúster Ceph RBD unificado

Clonado instantáneo (CoW)

Alta disponibilidad: Galera y Raft

De redes por software a alta disponibilidad

VLAN = red local virtual · OVN = red virtual abierta · DVR = enrutamiento distribuido · RBD = bloques de Ceph · HA = alta disponibilidad

SEMANA 13 · ARQUITECTURA EN LA NUBE

OpenStack II: Arquitectura Interna, Redes de Producción y Alta Disponibilidad

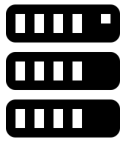
Parte 3 de 4 · Almacenamiento Distribuido Ceph y Alta Disponibilidad

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Parte 3 de 4 · Almacenamiento Distribuido Ceph y Alta Disponibilidad

Almacenamiento Efímero frente a Cinder

Comparativa entre discos en hipervisor local y volúmenes en red SAN/Ceph

Criterio	 Disco efímero	 Volumen Cinder
Dónde vive	En el hipervisor local	En clúster Ceph o SAN
Rendimiento	NVMe local, sin red	Usa la red de almacenamiento
Si cae el nodo	Se pierden instancia y datos	El volumen sobrevive
Uso típico	Aplicaciones sin estado	Datos críticos y migración

SAN = red de área de almacenamiento · NVMe = memoria no volátil express

Fuente: docs.openstack.org/cinder/latest/

Ceph RADOS y Algoritmo CRUSH

Cálculo matemático determinista de ubicación sin servidor central de metadatos



Réplica 3x en racks distintos

CRUSH = replicación controlada por hash · OSD = disco de Ceph · RADOS = almacén confiable de objetos distribuidos

Fuente: docs.ceph.com/en/latest/architecture/

Ceph RBD Unificado: Glance y Cinder

Un mismo backend distribuido para imágenes, volúmenes de datos y cómputo



Glance

imágenes



Cinder

volúmenes



Nova

discos efímeros

pool images

pool volumes

pool vms



Clúster Ceph RBD único

Almacenamiento por bloques distribuido

NVMe y HDD en un mismo clúster

Crear discos no mueve bytes

RBD = bloques de Ceph · HDD = disco duro

Fuente: docs.ceph.com/en/latest/rbd/rbd-openstack/

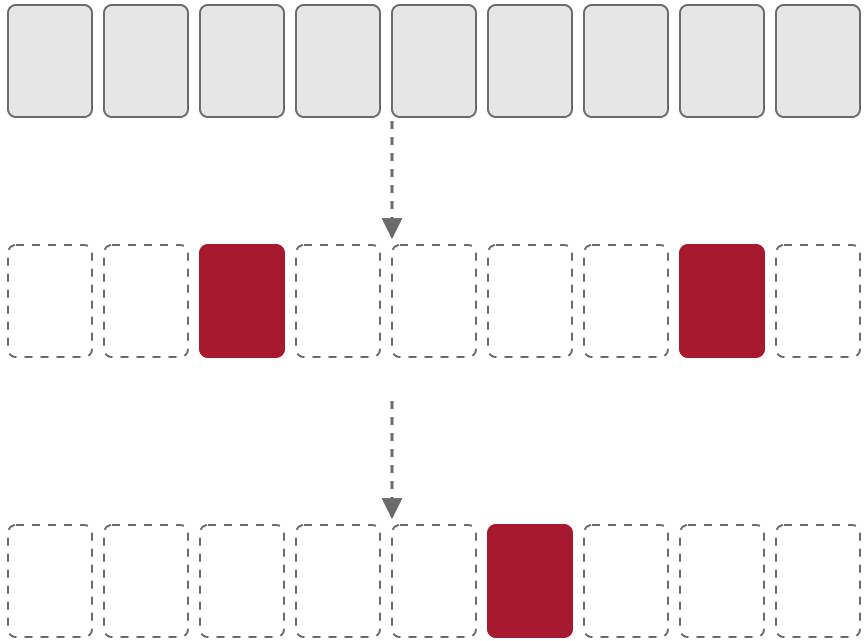
Clonación Instantánea: Copy-on-Write (CoW)

Discos nuevos al instante: solo se guardan los cambios

Imagen base Glance
solo lectura · Ubuntu (LTS)

Volumen VM 1
lee la base, escribe cambios

Volumen VM 2
lee la base, escribe cambios



Clonación casi instantánea

Qué se copia

Cada VM guarda solo sus cambios, no toda la imagen base.

Miles de VM sin copiar el archivo base por la red.

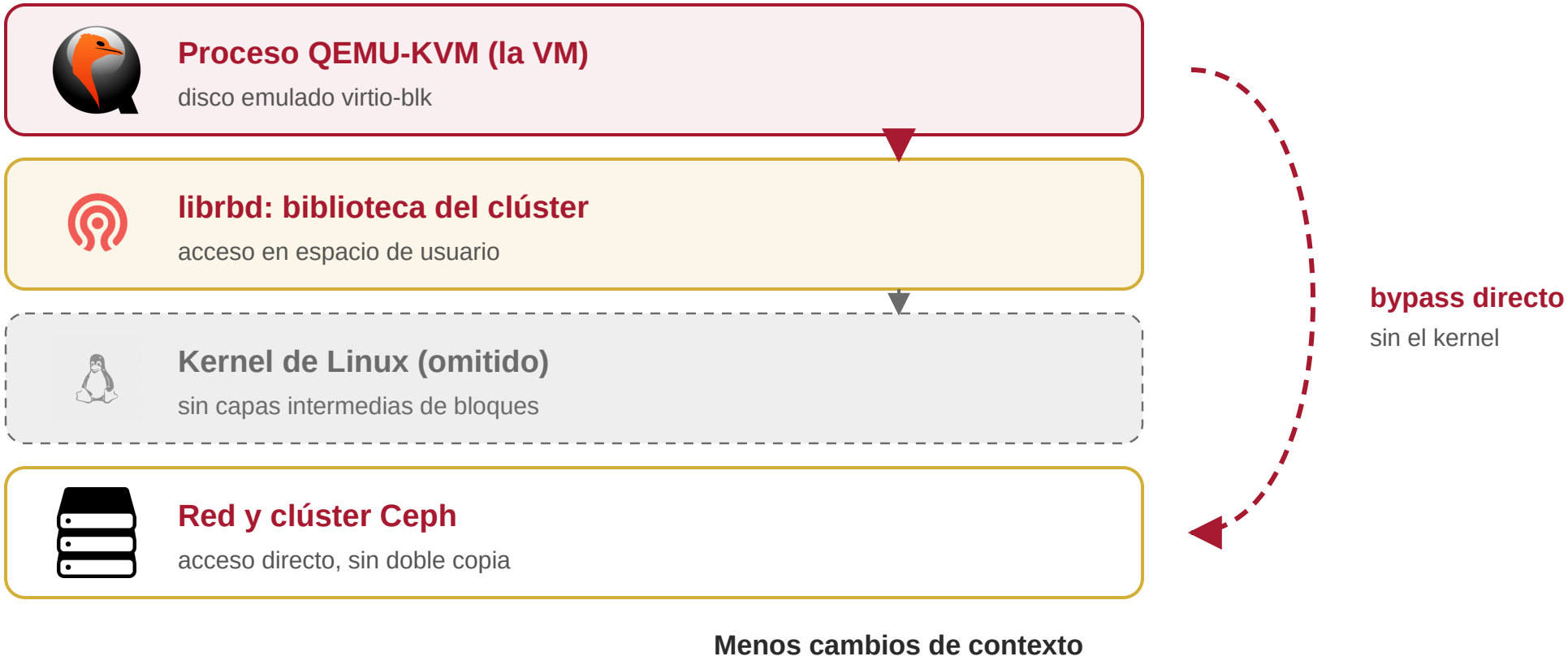
Gris: bloques compartidos
Vino: bloques nuevos (delta)

VM = máquina virtual · GB = gigabyte

Fuente: docs.ceph.com/en/latest/rbd/rbd-openstack/

Conexión Librbd: De Ceph al Proceso KVM

Eliminación de capas intermedias del kernel mediante acceso nativo en espacio de usuario



QEMU = emulador de hardware · KVM = virtualización en el kernel · VM = máquina virtual

Fuente: docs.openstack.org/cinder/latest/configuration/block-storage/drivers/ceph-rbd-volume-driver.html

Topología de Control de Tres Nodos

Quórum impar para sobrevivir a la pérdida de un controlador



Racks de un centro de datos (ilustración)

Quórum impar: 3 controladores

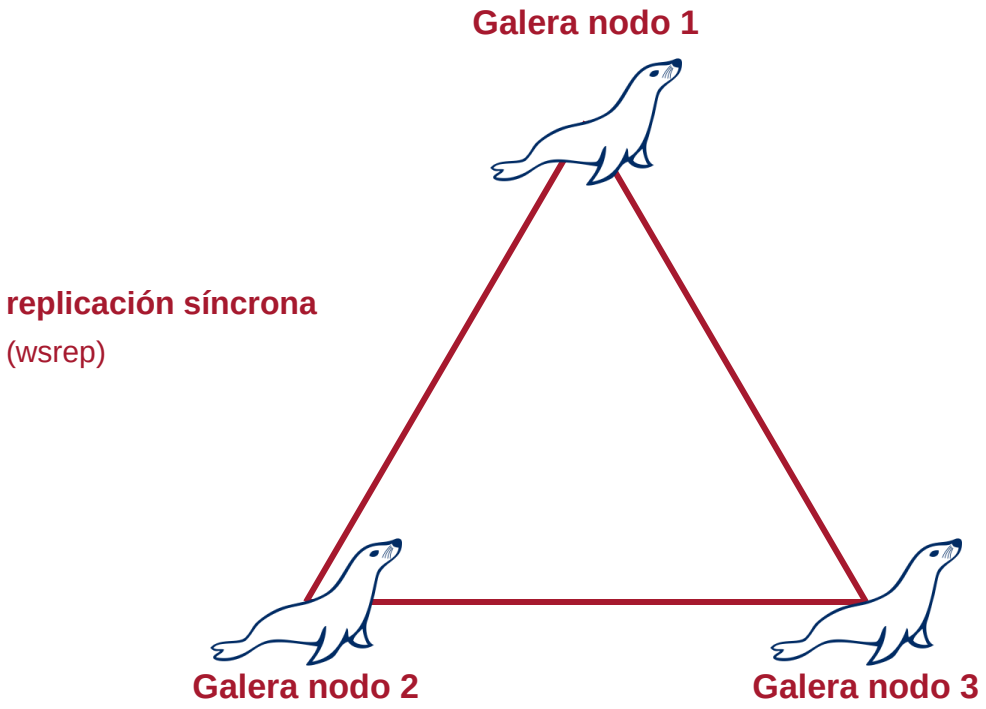
	Controlador 1	Controlador 2	Controlador 3
APIs OpenStack	▢	▢	▢
MariaDB Galera	🦭	🦭	🦭
RabbitMQ (quórum)	🐇	🐇	🐇

Quedan 2 de 3 votos: sigue activo

Fuente: OpenStack Architecture Design Guide · Foto: Victorgrigas, Wikimedia Commons (CC BY-SA 3.0)

MariaDB Galera: Replicación Síncrona

Replicación multi-master con certificación y consistencia ACID estricta



Certifica cada transacción

en todos los nodos

Confirma solo si todos

certifican: sin pérdida de datos

Copia idéntica en 3 controladores

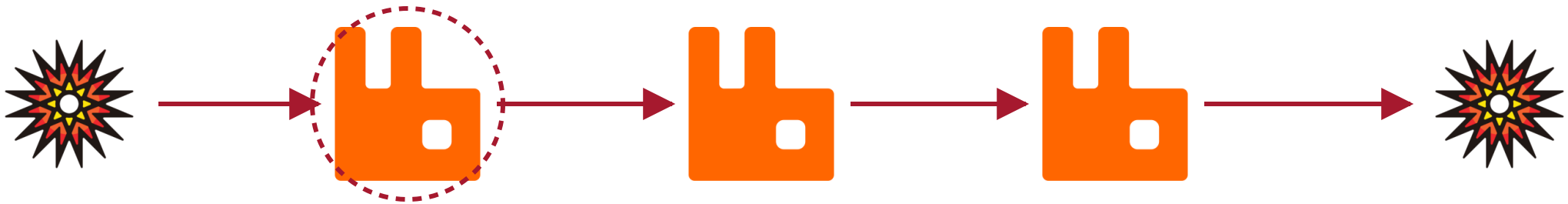
Replicación multi-master con consistencia ACID

ACID = atomicidad, consistencia, aislamiento y durabilidad

Fuente: galera-cluster.com/library/documentation/

RabbitMQ: Quorum Queues con Raft

Sustitución de colas reflejadas clásicas por consenso formal tolerante a fallas



Nova Conductor
publica el mensaje

Broker 1 (líder)
escribe en su registro

Brokers 2 y 3
siguen al líder (Raft)

Mayoría confirma
2 de 3 guardan en disco

Destino
recibe sin pérdidas

Quorum Queues con Raft (consenso por mayoría)

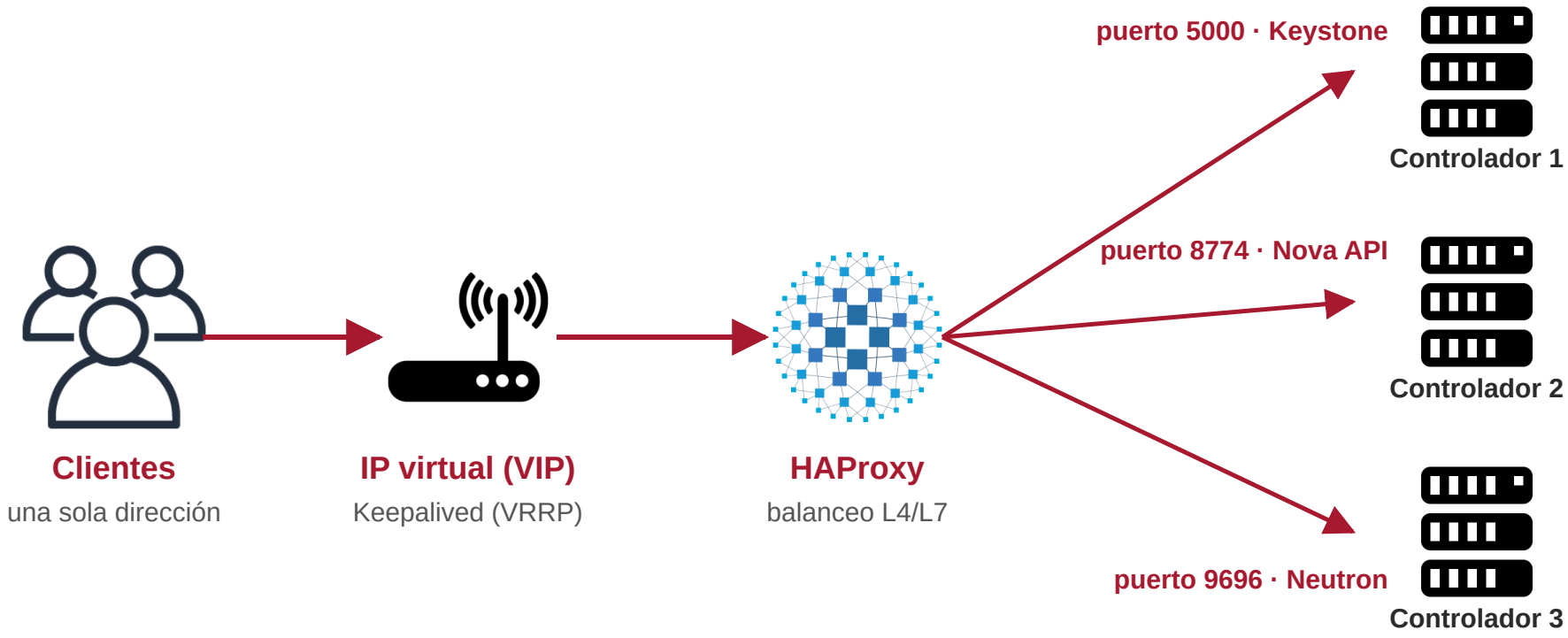
Reemplazan a las colas reflejadas clásicas

Otro líder toma el mando

Fuente: rabbitmq.com/quorum-queues.html

Front-End: Keepalived Virtual IP y HAProxy

Punto de entrada único con conmutación VRRP y balanceo de carga L4/L7



Si cae un balanceador, la IP virtual pasa al otro.

IP = dirección de red · L4 = capa 4 (transporte) · L7 = capa 7 (aplicación) · API = interfaz de programación

Fuente: docs.openstack.org/ha-guide/controller-ha.html

Aislamiento de Nodos con STONITH

Corte de energía por IPMI para evitar corrupción de datos por split-brain



Ceph ya no recibe escrituras

Mejor un servidor apagado que uno dudoso.

STONITH = apagar el otro nodo · IPMI = control remoto de la placa

Fuente: clusterlabs.org/pacemaker/doc/

Snapshots y Copias Consistentes

Sincronización mediante qemu-guest-agent antes de congelar bloques en storage

1. **Nova o Cinder piden la copia**
   orden de snapshot
2. **fsfreeze congela escrituras**
   qemu-guest-agent vacía buffers
3. **Ceph crea el snapshot CoW**
   copia instantánea y consistente
4. **fsthaw descongela la VM**
   las escrituras se reanudan
5. **Copia consistente lista**
   sin apagar la máquina virtual



qemu-guest-agent



snapshot CoW en Ceph

Sin esta coordinación, la copia puede quedar con transacciones a medias.

VM = máquina virtual

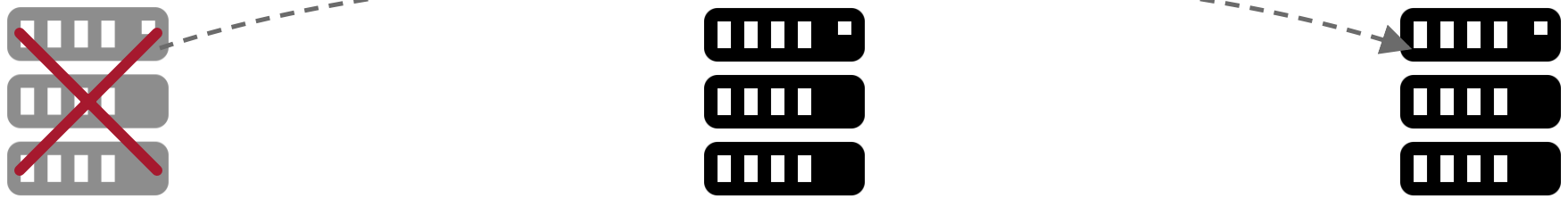
Fuente: docs.openstack.org/cinder/latest/admin/block-storage-volume-backups.html

CASO ILUSTRATIVO

Caso de Estudio: Resiliencia ante Pérdida de Nodo

Caso ilustrativo, no es una empresa real: tolerancia a fallas de un controlador

La IP virtual migra con VRRP



Controlador 1

caído (fallo eléctrico)

Controlador 2

activo: mantiene el quórum

Controlador 3

activo: asume la IP virtual

2 de 3 votos: quórum preservado

Cero caída de cargas

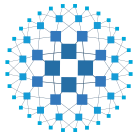
IP = dirección de red · VRRP = protocolo de redundancia de enrutador virtual

Fuente: Modelo arquitectónico de resiliencia en nube privada 2026

RESUMEN TECNOLÓGICO

Matriz de Alta Disponibilidad

Mecanismos y protocolos de tolerancia a fallas por cada componente

	Componente	Mecanismo	Sin punto único de falla (SPOF)
	**Base de datos (MariaDB)**	Galera: replicación síncrona en 3 nodos	✓
	**Mensajería (RabbitMQ)**	Quorum Queues con algoritmo Raft	✓
	**Acceso a las APIs**	IP virtual con Keepalived y HAProxy	✓
	**Almacenamiento (Ceph)**	CRUSH y réplica 3x entre racks	✓

Ninguna pieza queda sola: cada capa tiene respaldo.

IP = dirección de red · CRUSH = replicación controlada por hash

Fuente: OpenStack High Availability Guide 2026

SÍNTESIS Y PRÓXIMO PASO

Hacia la Parte 4: Despliegue y TCO

De la ingeniería de resiliencia hacia las decisiones estratégicas y de negocio

Contenido de la parte 4



DevStack vs Kolla



Privada vs pública



TCO a 3 años

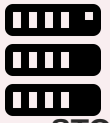
Dominado en la parte 3



Ceph y CoW



HA de 3 controladores



Galera y STONITH

De resistir fallas a decidir cómo operar y cuánto cuesta.

HA = alta disponibilidad · STONITH = apagar el otro nodo · TCO = costo total de propiedad

SEMANA 13 · ARQUITECTURA EN LA NUBE

OpenStack II: Arquitectura Interna, Redes de Producción y Alta Disponibilidad

Parte 4 de 4 · Operación en Producción, Despliegue y Viabilidad Económica

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Parte 4 de 4 · Operación en Producción, Despliegue y Viabilidad Económica

DevStack frente a Producción

Diferencias estructurales entre entornos de desarrollo y arquitecturas empresariales



DevStack

solo laboratorio

- un solo nodo (all-in-one)
- procesos directos en el host
- se reinicia al estado base



Kolla-Ansible

producción

- 3 controladores (alta disponibilidad)
- contenedores Docker inmutables
- gestión declarativa con Ansible

Docker (contenedores) · Ansible (automatización)

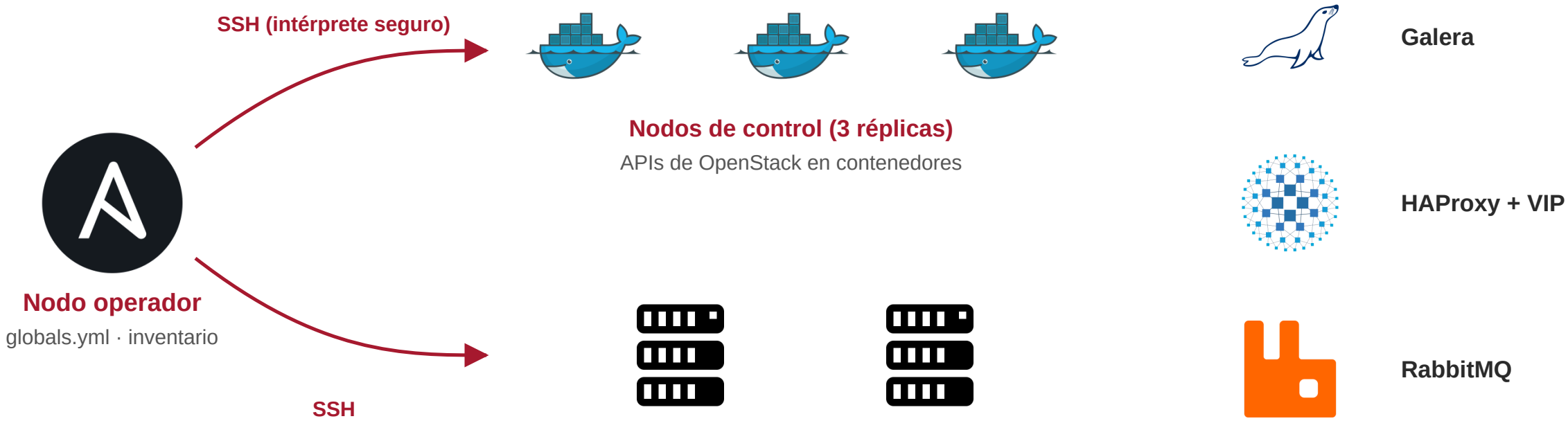


Racks de producción

Fuente: docs.openstack.org/kolla-ansible/latest/ · Foto: NOIRLab/NSF/AURA, T. Slovinský (CC BY 4.0)

Arquitectura de Kolla-Ansible

Flujo de despliegue desde el nodo operador hacia los nodos del clúster



Cada servicio viene en su contenedor; el mismo inventario lo reconstruye.

SSH = intérprete seguro · QEMU = emulador de hardware · OVN = red virtual abierta · VIP = dirección virtual compartida

Fuente: docs.openstack.org/kolla-ansible/latest/user/quickstart.html

Segmentación de Redes Físicas

Separación de tráfico en interfaces de red dedicadas o VLANs troncales

Switches ToR con LACP (agregación de enlaces)



1. **VLAN 10 · gestión**
   APIs, Galera y SSH
2. **VLAN 20 · overlay**
   Geneve y MTU jumbo 9000
3. **VLAN 30 · almacenamiento**
   Ceph RBD y replicación
4. **VLAN 40 · externa / provider**
   IPs flotantes y salida

ToR = switch del rack · LACP = agregación de enlaces · VLAN = red local virtual · SSH = intérprete seguro · MTU = unidad máxima de transmisión · RBD = bloques de Ceph

Fuente: docs.openstack.org/install-guide/ · Foto: Dsimic, Wikimedia Commons (CC BY-SA 4.0)

Migración en Vivo (Live Migration)

Transferencia en caliente del estado de CPU y RAM entre hipervisores KVM



La VM no se apaga

VM = máquina virtual · RAM = memoria de acceso aleatorio · vCPU = núcleo virtual asignado a una instancia · RBD = bloques de Ceph · CPU = unidad central de procesamiento · KVM = virtualización en el kernel

Fuente: docs.openstack.org/nova/latest/admin/live-migration-usage.html

Actualizaciones Continuas (Rolling Upgrades)

Actualización gradual nodo a nodo manteniendo compatibilidad de protocolo N-1

Compatibilidad N y N+1



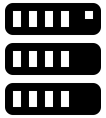
Base de datos

1. Expansión de BD



Controladores

2. Rolling: nodo 1 → 2 → 3



Hipervisores

3. Drenado y contracción

tiempo

Los hipervisores migran sus VM antes

BD = base de datos · VM = máquina virtual

Fuente: docs.openstack.org/nova/latest/user/upgrade.html

Modelos Financieros: CapEx frente a OpEx

Inversión de capital fija frente a costos operativos basados en consumo elástico

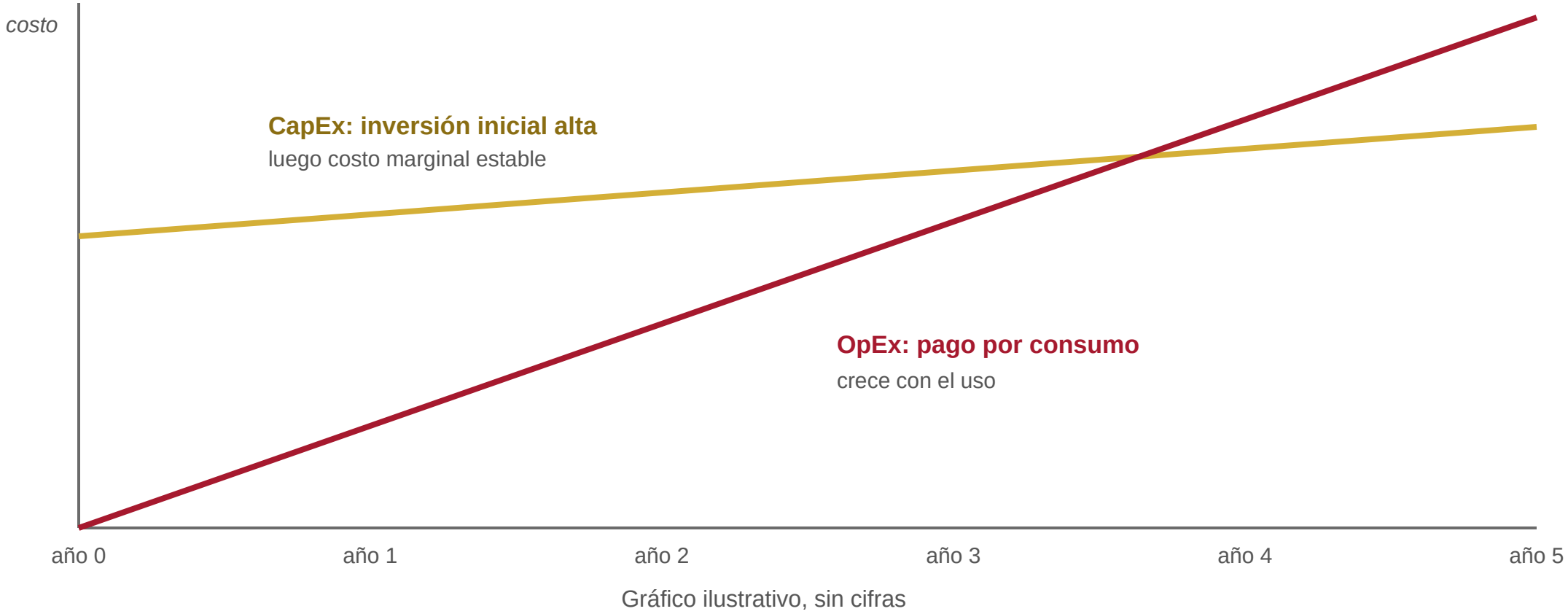


Gráfico ilustrativo, sin cifras

Fuente: OpenInfra Foundation Whitepaper: Private Cloud Economics

CRITERIOS DE DECISIÓN

¿Cuándo Conviene una Nube Privada?

Parámetros técnicos y normativos que justifican operar infraestructura propia



Sala de cómputo propia

1. **Cargas estables y predecibles**
   utilización sostenida alta
2. **Soberanía de datos**
   llaves en HSM (módulo de seguridad)
3. **Latencia ultra-baja local**
   sin depender de la WAN (red amplia)
4. **Volumen masivo de datos**
   sin cargos de salida (egress)

HSM = módulo de seguridad · WAN = red amplia

Fuente: OpenInfra Foundation: Enterprise Cloud Strategy · Foto: Rubin Observatory/NSF/AURA (CC BY 4.0)

Estructura del Costo Total de Propiedad (TCO)

Costo a tres años: nube privada frente a pública

TCO (costo total de propiedad) = Hardware + Energía + Red + Operación + Licencias



Nube privada (CapEx + OpEx)

Hardware	Instalaciones	Personal (SRE)



Nube pública (OpEx)

Cómputo bajo demanda	Volúmenes EBS	Tráfico de salida

Proporciones ilustrativas

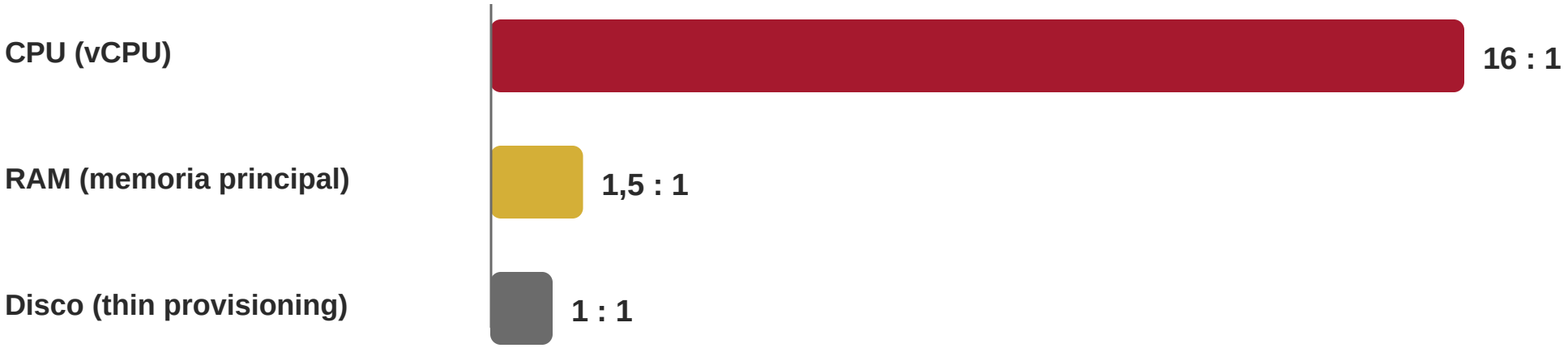
EBS = volúmenes de bloques elásticos

Fuente: Gartner Research: How to Calculate Infrastructure TCO

Sobresuscripción Segura de Recursos

Ratios recomendados de sobreasignación para maximizar densidad sin degradación

Valores por defecto de Nova (documentación oficial)



Ajustar según la carga

- CPU: sobreasignar es normal
- RAM: no sobreasignar (swap)
- Disco: thin provisioning

CPU = unidad central de procesamiento · RAM = memoria de acceso aleatorio

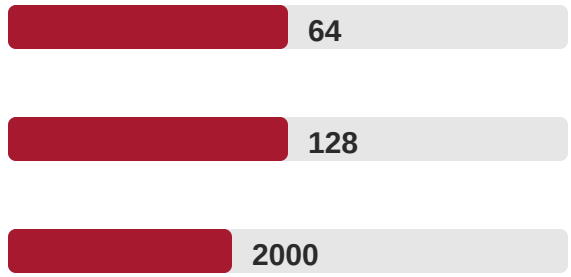
Fuente: docs.openstack.org/nova/latest/admin/scheduling.html

Multitenancy y Cuotas en Keystone

Aislamiento lógico, dominios organizacionales y límites de consumo por proyecto

Dominio corporativo (Keystone domain)

Proyecto: finanzas



Proyecto: analítica y datos



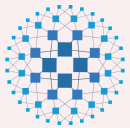
Ejemplo: la API rechaza lo que excede

vCPU = núcleo virtual asignado a una instancia · RAM = memoria de acceso aleatorio · API = interfaz de programación

Fuente: docs.openstack.org/keystone/latest/admin/identity-concepts.html

Hardening de la Infraestructura OpenStack

Capas defensivas en APIs, transporte, aislamiento del hipervisor y auditoría



1. Cifrado TLS

TLS en HAProxy y mTLS entre servicios



2. Aislamiento

sVirt y SELinux; QEMU en sandbox



3. Control de acceso

tokens de 1 hora · RBAC



4. Auditoría

eventos CADF hacia SIEM

TLS = seguridad de capa de transporte · QEMU = emulador de hardware · RBAC = control por roles · CADF = formato de auditoría · SIEM = gestión de eventos de seguridad

Fuente: docs.openstack.org/security-guide/

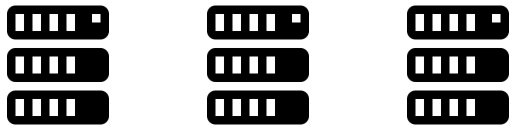
Topología de Referencia para Producción

Caso ilustrativo de arquitectura, no es una empresa real

Conmutación redundante Top of Rack (ToR)

bonds LACP: agregación de enlaces

3 nodos controladores HA



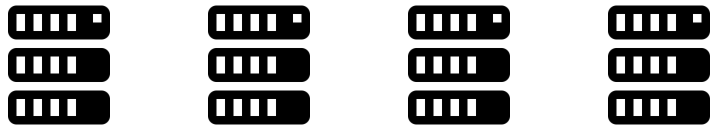
VIP + Keepalived + HAProxy

APIs Nova, Neutron, Cinder

MariaDB Galera (3 réplicas)

RabbitMQ y OVN (Raft)

Nodos de cómputo KVM



Hipervisores KVM y QEMU

ovn-controller local (OVN)

DVR: enrutamiento local

vCPU 3:1 y RAM 1:1 (ejemplo)

Clúster Ceph distribuido



3 MON y MGR (monitores)

Pools RBD para Cinder

Pool Glance para imágenes

Réplica 3x entre OSD

Caso ilustrativo de arquitectura, no es una empresa real.

LACP = agregación de enlaces · HA = alta disponibilidad · VIP = dirección virtual compartida · KVM = virtualización en el kernel · QEMU = emulador de hardware · DVR = enrutamiento distribuido · vCPU = núcleo virtual asignado a una instancia · RAM = memoria de acceso aleatorio · MON = monitor del clúster · MGR = gestor del clúster · RBD = bloques de Ceph · OSD = disco de Ceph

Fuente: OpenStack Architecture Design Guide (docs.openstack.org)

Interconexión Híbrida con AWS

Extensión de la nube privada hacia servicios elásticos mediante enlaces dedicados



Cloud bursting: solo en los picos

Sin Internet pública: datos en Ceph

BGP = protocolo de pasarela de frontera · VPC = red virtual privada

Fuente: AWS Direct Connect (Amazon Web Services) / OpenStack Hybrid Cloud Architecture

PRÁCTICA GUIADA

Transición al Laboratorio OpenStack

Consolidación de conceptos teóricos mediante la práctica en el entorno de laboratorio



1 **Plano de control**

Keystone, Nova y Glance en consola

2 **Redes y máquinas virtuales**

subredes de proyecto e IP flotantes

Guía de práctica en el portal

laboratorios/lab-openstack/

IP = dirección de red

Fuente: Guía de Laboratorio OpenStack · Foto: PiDatacenters, Wikimedia Commons (CC BY-SA 4.0)